Real-world (part 2) — run in parallel, fall back, and stream — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `lcel_patterns_practice.py`: once anything is an LCEL `Runnable` (which everything built with `|` is), three powerful features come for free, and this exercise uses each one once to prove it. `RunnableParallel` runs several chains at the same time and hands you back a dict of their results — so two independent lookups take as long as the slower one, not the sum of both. `.with_fallbacks()` wraps a chain with a backup that's used only if the first one raises — so one provider having an outage doesn't take your whole feature down. `.stream()` yields the answer in pieces as the model produces them, instead of making the user wait for the entire reply. Seeing all three on a tiny chain here is what makes them obvious when they're doing real work later. **If not:** Project 6's parallel lookups, fallbacks and streamed replies would be the first time you met `RunnableParallel`, `.with_fallbacks()` and `.stream()`.

Solution

Read it top to bottom — each of the four LCEL patterns is its own short, commented section.

```
# lcel_patterns_practice.py
import time

import openai
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda, RunnableParallel
from langchain_openai import ChatOpenAI


# 1) RUN IN PARALLEL ------------------------------------------------------
# Both lookups get the same input and run at the same time. Each sleep
# stands in for a slow network call.
def get_weather(inputs):
    time.sleep(1)
    return "sunny in " + inputs["city"]


def get_news(inputs):
```

```
    time.sleep(1)
    return "a new park opened in " + inputs["city"]


lookups = RunnableParallel({"weather": get_weather, "news": get_news})

start = time.time()
result = lookups.invoke({"city": "Lahore"})
seconds = time.time() - start
print(f"case 1 (keys)         -> {sorted(result.keys())}")
print(f"case 1 (under 1.5 s?) -> {seconds < 1.5}")


# 2) FALL BACK ------------------------------------------------------------
# If the model call fails, use a backup reply instead of crashing.
def backup_reply(inputs):
    return "Sorry, the assistant is busy. Please try again soon."


prompt = ChatPromptTemplate.from_template("Write one sentence about {city}.")

# nothing listens on port 9, so this model call always fails (a forced outage)
broken_model = ChatOpenAI(
    model="gpt-4o-mini", base_url="http://127.0.0.1:9", max_retries=0
)
broken_chain = (prompt | broken_model | StrOutputParser()).with_fallbacks(
    [RunnableLambda(backup_reply)],
    # fall back only on real outages, so a bug in your code still fails loudly
    exceptions_to_handle=(openai.APIConnectionError, openai.RateLimitError),
)
reply = broken_chain.invoke({"city": "Lahore"})
print(f"case 2 (outage)       -> backup used: {reply.startswith('Sorry')}")


# 3) A CODE BUG MUST STILL FAIL LOUDLY ------------------------------------
# A KeyError is a bug in your own code, not a transient outage, so the
# backup above must NOT hide it.
def buggy_step(inputs):
    return inputs["missing_key"]


bug_chain = RunnableLambda(buggy_step).with_fallbacks(
    [RunnableLambda(backup_reply)],
    exceptions_to_handle=(openai.APIConnectionError, openai.RateLimitError),
)
try:
    bug_chain.invoke({"city": "Lahore"})
```

```
    print("case 3 (bug)          -> hidden (wrong!)")
except KeyError:
    print("case 3 (bug)          -> KeyError still raised")


# 4) STREAM ------------------------------------------------------------
# .stream() hands back the reply in small pieces; print each as it arrives.
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
live_chain = prompt | model | StrOutputParser()

print("case 4 (stream)       -> ", end="")
full_text = ""
for chunk in live_chain.stream({"city": "Lahore"}):
    print(chunk, end="", flush=True)
    full_text = full_text + chunk
print()
print(f"case 4 (full text)    -> {len(full_text)} characters")
```

Expected output (the streamed sentence varies):

```
case 1 (keys)          -> ['news', 'weather']
case 1 (under 1.5 s?)  -> True
case 2 (outage)        -> backup used: True
case 3 (bug)           -> KeyError still raised
case 4 (stream)        -> Lahore is a busy, historic city in Pakistan.
case 4 (full text)     -> 44 characters
```